

Lab Sheet: 06026212 DATA WAREHOUSING

(การสร้างคลังข้อมูล)

สัปดาห์ที่ 3: การวิเคราะห์ Requirement และการสร้าง KPI Layer ด้วย dbt

กรณีศึกษา: DVD Rental | ระยะเวลา: 2 ชั่วโมง

วัตถุประสงค์

1. วิเคราะห์ Subject Area, Business Process และคำถามทางธุรกิจจากกรณีศึกษา dvdrental ได้
2. กำหนด KPI, สูตรคำนวณ, หน่วยวัด และ Granularity ที่ชัดเจนได้
3. อธิบายปัญหาการนับซ้ำที่เกิดจากการ Join ตาราง payment กับ rental ได้
4. สร้าง dbt project และพัฒนา Staging, Intermediate, Fact และ Metric Models ได้
5. สร้าง Metric Key Catalog ด้วย dbt seed เพื่อให้ KPI มีรหัสและนิยามมาตรฐาน
6. ตรวจสอบความถูกต้องของข้อมูลด้วย dbt tests และนำ Metric Models ไปสร้าง Dashboard บน Metabase ได้

เครื่องมือที่ใช้

เครื่องมือ	ใช้ทำอะไร
Docker Compose	รัน PostgreSQL, pgAdmin, dbt, Metabase และบริการอื่นใน environment เดียวกัน
PostgreSQL	เก็บฐานข้อมูลต้นทาง dvdrental และผลลัพธ์ที่ dbt สร้าง
dbt Core	จัดการ SQL transformation, dependency, seed, test และสร้าง KPI Layer
pgAdmin	ตรวจสอบฐานข้อมูล ตาราง View และผลลัพธ์ของ dbt
Metabase	สร้าง Dashboard จาก Metric Models โดยไม่เขียนสูตร KPI ซ้ำ
Text Editor / VS Code	สร้างและแก้ไขไฟล์ .sql, .yaml และ .csv ใน dbt project

Dataset และเงื่อนไขก่อนเริ่ม Lab

ใช้ฐานข้อมูลตัวอย่าง PostgreSQL ชื่อ dvdrental ซึ่งนำเข้าไว้แล้วจาก Lab 2 โดยตารางหลักที่ใช้ใน Lab นี้ประกอบด้วย rental, payment, inventory และ film

- มีโฟลเดอร์ DWH_Lab และไฟล์ docker-compose.yaml จากชุด DWH_Lab(2).zip
- เคยนำเข้า dvdrental ลง PostgreSQL แล้ว หากยังไม่มีให้ย้อนกลับไปทำขั้นตอน Restore จาก Lab 2
- เปิด Docker Desktop และตรวจสอบว่า Docker Engine ทำงานอยู่
- เปิด Terminal หรือ PowerShell ที่โฟลเดอร์เดียวกับ docker-compose.yaml

```
docker exec -it dbt bash
```

กรณีศึกษาและ Requirement ที่กำหนดให้

DVD Rental Network มีร้านเช่าภาพยนตร์หลายสาขา ผู้บริหารพบว่ารายงานจากแต่ละฝ่ายให้ตัวเลขไม่ตรงกัน เพราะแต่ละทีมเขียนสูตร KPI เอง เช่น บางทีมใช้จำนวน payment เป็นจำนวนครั้งเช่า ขณะที่อีกทีมใช้จำนวน rental ส่งผลให้ KPI เดียวกันมีหลายคำตอบ

ทีม Data Warehouse จึงต้องกำหนด KPI กลาง สร้างรหัส Metric Key ที่อ้างอิงได้แน่นอน และให้ Metabase อ่านค่าจาก KPI Layer ที่ dbt สร้างแทนการคำนวณสูตรใหม่ใน Dashboard

1. Subject Area และ Business Process

Subject Area	Business Process	เหตุการณ์ที่ทำให้เกิดข้อมูล
Rental and Revenue Performance	Rent a Film	ลูกค้าเช่าภาพยนตร์หนึ่งรายการและเกิด rental_id
Rental and Revenue Performance	Return a Film	ลูกค้านำภาพยนตร์กลับมาคืนและระบบบันทึก return_date
Rental and Revenue Performance	Receive Payment	พนักงานรับชำระเงินและระบบบันทึก amount ใน payment

2. คำถามทางธุรกิจและ KPI

Metric Key	KPI	คำถามที่ต้องตอบ	นิยามและสูตร
M001	Total Revenue	บริษัทมีรายได้รวมเท่าใด	SUM(revenue_amount)
M002	Rental Count	มีการเช่าภาพยนตร์กี่ครั้ง	COUNT ของ rental_id ที่ grain หนึ่งแถวต่อหนึ่ง rental
M003	Active Customer Count	มีลูกค้าที่มาใช้บริการกี่คน	COUNT DISTINCT(customer_id) ภายในช่วงเวลาที่เราวิเคราะห์
M004	Average Revenue per Rental	รายได้เฉลี่ยต่อการเช่าหนึ่งครั้งเท่าใด	M001 / M002
M005	Late Return Rate	รายการที่คืนล่าช้าคิดเป็นสัดส่วนเท่าใด	จำนวนรายการคืนล่าช้า / จำนวนรายการที่คืนแล้ว

นิยามการคืนล่าช้า: ถือว่าคืนล่าช้าเมื่อ return_date มากกว่า rental_date บวก rental_duration วัน และตัวหารของ Late Return Rate ใช้เฉพาะรายการที่คืนแล้ว เพื่อไม่ให้รายการที่ยังไม่คืนถูกจัดเป็นการคืนตรงเวลาหรือคืนล่าช้าโดยอัตโนมัติ

3. Grain Statement

Model	Grain Statement
fct_rental_activity	1 แถว = การเช่าภาพยนตร์ 1 ครั้ง หรือ 1 rental_id
metric_company_monthly	1 แถว = 1 เดือน × 1 metric_key ของทั้งบริษัท
metric_store_monthly	1 แถว = 1 เดือน × 1 สาขา × 1 metric_key

การกำหนด Grain ก่อนเขียน SQL ช่วยป้องกัน Double Counting โดยเฉพาะกรณีที่ payment มีหลายแถวต่อ rental_id เดียว จึงต้อง Aggregate payment ให้เหลือหนึ่งแถวต่อ rental_id ก่อนนำไป Join กับ rental

4. Bus Matrix

Business Process	Date	Customer	Film	Store	Staff
Rent a Film	X	X	X	X	X
Return a Film	X	X	X	X	
Receive Payment	X	X		X	X

ส่วนที่ 1: สร้างโครงสร้าง dbt Project

ในโฟลเดอร์ DWH_Lab ให้สร้างโฟลเดอร์และไฟล์ตามโครงสร้างต่อไปนี้ โดยใช้ File Explorer หรือ VS Code ก็ได้

```

DWH_Lab/
├── docker-compose.yaml
├── dbt/
│   ├── dvd_kpi
│   │   ├── dbt_project.yml
│   │   └── models/
│   │       ├── sources.yml
│   │       ├── schema.yml
│   │       └── staging/
│   │           ├── stg_rental.sql
│   │           ├── stg_payment.sql
│   │           ├── stg_inventory.sql
│   │           └── stg_film.sql
│   │       ├── intermediate/
│   │           └── int_payment_by_rental.sql
│   │       └── marts/
│   │           ├── fct_rental_activity.sql
│   │           ├── metric_company_monthly.sql
│   │           └── metric_store_monthly.sql
│   ├── seeds/
│   │   └── metric_definition.csv
│   ├── tests/
│   │   ├── assert_metric_company_grain.sql
│   │   ├── assert_metric_store_grain.sql
│   │   └── assert_late_return_rate_range.sql
│   └── dbt_root/
│       └── profiles.yml

```

โฟลเดอร์ dbt ถูก mount เข้า container ที่ /usr/app ส่วน dbt_root ถูก mount เข้า /root/.dbt ตาม docker-compose.yaml ดังนั้นไฟล์ที่สร้างบนเครื่องจะมองเห็นได้จาก dbt container ทันที

ส่วนที่ 2: ตั้งค่า dbt Project และ Connection

2.1 สร้างไฟล์ dbt/dbt_project.yml

```
name: dvd_kpi
version: '1.0.0'
config-version: 2

profile: dvd_kpi

model-paths: ['models']
seed-paths: ['seeds']
test-paths: ['tests']

clean-targets:
  - 'target'
  - 'dbt_packages'

models:
  dvd_kpi:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: view
      +schema: intermediate
    marts:
      +materialized: view
      +schema: metrics

seeds:
  dvd_kpi:
    +schema: metadata
  metric_definition:
    +column_types:
      metric_key: varchar(10)
```

คำอธิบายส่วนสำคัญ:

- name คือชื่อ dbt project และต้องตรงกับชื่อได้ models และ seeds
- profile ระบุชื่อ connection profile ที่ dbt จะไปอ่านจาก profiles.yml
- model-paths, seed-paths และ test-paths ระบุตำแหน่งไฟล์ประเภทต่าง ๆ
- +materialized: view กำหนดให้ dbt สร้าง PostgreSQL View แทน Table
- +schema กำหนด schema แยกตามชั้นข้อมูล โดย dbt จะต่อท้าย target schema เช่น dbt_metrics
- seed metric_definition จะถูกโหลดเป็นตาราง metadata และกำหนด metric_key เป็น varchar(10)

2.2 สร้างไฟล์ dbt_root/profiles.yml

```
dvd_kpi:
  target: dev
  outputs:
    dev:
      type: postgres
      host: postgres
      port: 5432
      user: dw_user
      password: dw_pass
      dbname: dvdrental
      schema: dbt
      threads: 4
```

คำอธิบายส่วนสำคัญ:

- host: postgres ใช้ชื่อ service ภายใน Docker network ห้ามใช้ localhost เพราะ localhost ภายใน dbt container หมายถึงตัว dbt container เอง
- port: 5432 เป็น port ภายใน Docker network ไม่ใช่ port 25432 ที่เปิดให้เครื่อง host

- dbname: dvdrental คือฐานข้อมูลต้นทางและปลายทางของ Lab
- schema: dbt เป็น target schema หลัก จึงได้ schema ปลายทาง เช่น dbt_staging, dbt_intermediate, dbt_metrics และ dbt_metadata
- threads: 4 อนุญาตให้ dbt ประมวลผลได้พร้อมกันสูงสุด 4 งาน

2.3 ทดสอบ Connection

```
cd dvd_kpi
dbt debug
```

dbt debug ตรวจสอบว่า dbt_project.yml, profiles.yml, adapter และการเชื่อมต่อ PostgreSQL ถูกต้อง ผลลัพธ์ตอนท้ายควรแสดงข้อความลักษณะ All checks passed

ส่วนที่ 3: ประกาศ Source Tables

3.1 สร้างไฟล์ dbt/models/sources.yml

```
version: 2

sources:
  - name: dvdrental
    database: dvdrental
    schema: public

    tables:
      - name: rental
      - name: payment
      - name: inventory
      - name: film
```

คำอธิบาย:

- source ใช้ประกาศว่าตารางใดเป็นข้อมูลต้นทางที่ dbt ไม่ได้เป็นผู้สร้าง
- name: dvdrental เป็นชื่ออ้างอิงภายใน dbt ไม่จำเป็นต้องตรงกับชื่อฐานข้อมูล แต่กำหนดให้ตรงเพื่ออ่านง่าย
- database และ schema ที่ไปยังตารางจริงใน PostgreSQL
- เมื่อเขียน SQL จะอ้างอิงด้วย {{ source('dvdrental', 'rental') }} แทนการ hardcode public.rental

3.2 ตรวจสอบว่า dbt อ่าน Project ได้

```
dbt parse
```

dbt parse อ่านและตรวจสอบโครงสร้าง YAML, Jinja และ dependency โดยยังไม่สร้าง View หากไฟล์ YAML ยังไม่ถูกต้อง dbt จะแจ้งตำแหน่งที่เกิดปัญหา

ส่วนที่ 4: สร้าง Staging Models

Staging Models ทำหน้าที่เลือกคอลัมน์ที่ต้องใช้ แปลงชนิดข้อมูล และตั้งชื่อให้สม่ำเสมอ โดยยังไม่คำนวณ KPI

4.1 dbt/models/staging/stg_rental.sql

```
select
  rental_id::integer    as rental_id,
  rental_date::timestamp as rental_date,
  inventory_id::integer as inventory_id,
  customer_id::integer  as customer_id,
  return_date::timestamp as return_date,
  staff_id::integer     as staff_id
from {{ source('dvdrental', 'rental') }}
```

Model นี้เก็บเหตุการณ์การเช่าและการคืน โดย rental_id จะเป็นตัวกำหนด Grain ของ Fact Model

4.2 dbt/models/staging/stg_payment.sql

```
select
  payment_id::integer    as payment_id,
  customer_id::integer   as customer_id,
  staff_id::integer      as staff_id,
  rental_id::integer     as rental_id,
  amount::numeric(12, 2) as amount,
  payment_date::timestamp as payment_date
from {{ source('dvdrental', 'payment') }}
```

Model นี้เก็บการรับชำระเงิน โดย amount เป็นค่าที่ใช้สร้าง Total Revenue

4.3 dbt/models/staging/stg_inventory.sql

```
select
  inventory_id::integer as inventory_id,
  film_id::integer      as film_id,
  store_id::integer     as store_id
from {{ source('dvdrental', 'inventory') }}
```

inventory ทำหน้าที่เชื่อม rental กับ film และ store เพราะ rental ไม่มี film_id และ store_id โดยตรง

4.4 dbt/models/staging/stg_film.sql

```
select
  film_id::integer    as film_id,
  title::varchar      as film_title,
  rental_duration::integer as rental_duration,
  rating::varchar     as rating
from {{ source('dvdrental', 'film') }}
```

rental_duration คือจำนวนวันที่อนุญาตให้เช่า ใช้คำนวณ expected_return_datetime และ Late Return Rate

4.5 สร้าง Staging Views

```
dbt run --select stg_rental stg_payment stg_inventory stg_film
```

คำอธิบาย: dbt run สร้างเฉพาะ models ที่ระบุหลัง --select ผลลัพธ์จะอยู่ใน schema dbt_staging ตามการตั้งค่าใน dbt_project.yml

ส่วนที่ 5: Aggregate Payment ให้ตรงกับ Grain

5.1 ตรวจสอบความเสี่ยง Double Counting

ก่อน Join payment กับ rental ต้องตรวจสอบว่าหนึ่ง rental_id อาจมี payment ที่แถว หากมีมากกว่าหนึ่งแถวแล้ว Join โดยตรง Fact Model จะมีหลายแถวต่อ rental_id และทำให้ Rental Count ถูกนับซ้ำ

- รัน query ใน dw_pgadmin

```
SELECT rental_id, COUNT(*) AS payment_rows
FROM payment
WHERE rental_id IS NOT NULL
GROUP BY rental_id
HAVING COUNT(*) > 1
ORDER BY payment_rows DESC
LIMIT 10;
```

หาก Query คืนผลลัพธ์ แสดงว่ามี rental_id ที่สัมพันธ์กับ payment มากกว่าหนึ่งแถว จึงต้อง Aggregate ก่อน Join แม้ชุดข้อมูลบางเวอร์ชันจะไม่พบกรณีดังกล่าว การออกแบบนี้ยังคงปลอดภัยต่อข้อมูลในอนาคต

5.2 สร้าง dbt/models/intermediate/int_payment_by_rental.sql

```
select
  rental_id,
  sum(amount)::numeric(12, 2) as revenue_amount,
  count(*)::integer as payment_record_count
from {{ ref('stg_payment') }}
where rental_id is not null
group by rental_id
```

คำอธิบาย:

- ref('stg_payment') ทำให้ dbt สร้าง dependency และรู้ว่าต้องสร้าง stg_payment ก่อน
- GROUP BY rental_id ทำให้ผลลัพธ์เหลือหนึ่งแถวต่อหนึ่ง rental_id
- SUM(amount) คือรายได้รวมของ rental นั้น
- payment_record_count ใช้ตรวจสอบว่ามี payment ที่แถวถูก Aggregate เข้าด้วยกัน

```
docker compose exec dbt dbt run --select int_payment_by_rental
```

ส่วนที่ 6: สร้าง Fact Model

6.1 สร้าง dbt/models/marts/fct_rental_activity.sql

```

with rental as (
    select * from {{ ref('stg_rental') }}
),

inventory as (
    select * from {{ ref('stg_inventory') }}
),

film as (
    select * from {{ ref('stg_film') }}
),

payment_by_rental as (
    select * from {{ ref('int_payment_by_rental') }}
)

select
    r.rental_id,
    r.rental_date,
    date_trunc('month', r.rental_date)::date as rental_month,
    r.return_date,
    r.customer_id,
    r.staff_id,
    i.store_id,
    i.film_id,
    f.film_title,
    f.rating,
    f.rental_duration,

    r.rental_date
    + (f.rental_duration * interval '1 day')
    as expected_return_datetime,

    coalesce(p.revenue_amount, 0)::numeric(12, 2)
    as revenue_amount,

    coalesce(p.payment_record_count, 0)::integer
    as payment_record_count,

    1::integer as rental_count,

    case
        when r.return_date is not null then 1
        else 0
    end::integer as returned_rental_count,

    case
        when r.return_date is not null
        and r.return_date >
            r.rental_date
            + (f.rental_duration * interval '1 day')
        then 1
        else 0
    end::integer as late_rental_count

from rental r
join inventory i
    on r.inventory_id = i.inventory_id
join film f
    on i.film_id = f.film_id
left join payment_by_rental p
    on r.rental_id = p.rental_id

```

คำอธิบาย Logic สำคัญ:

- LEFT JOIN payment_by_rental ทำให้ rental ที่ยังไม่มี payment ไม่หายจาก Fact Model

- COALESCE(revenue_amount, 0) เปลี่ยนค่า NULL เป็น 0 สำหรับรายการที่ไม่มี payment
- rental_count กำหนดเป็น 1 เพราะหนึ่งแถวหมายถึงหนึ่ง rental จึงสามารถ SUM เพื่อหาจำนวนการเช่าได้
- returned_rental_count เป็น 1 เฉพาะรายการที่มี return_date
- late_rental_count เป็น 1 เมื่อ return_date เกิน expected_return_datetime
- ไม่กำหนดรายการที่ยังไม่คืนเป็น late โดยอัตโนมัติ เพราะ Requirement วัด Late Return Rate จากรายการที่คืนแล้ว

6.2 สร้าง Fact View

```
dbt run --select fct_rental_activity
```

dbt จะสร้าง View ชื่อ dbt_metrics.fct_rental_activity เนื่องจาก model อยู่ในโฟลเดอร์ marts และกำหนด +schema: metrics

ส่วนที่ 7: สร้าง Metric Key Catalog ด้วย dbt Seed

ใน Lab นี้ Metric Key หมายถึงรหัสอ้างอิง KPI ที่คงที่ เช่น M001 หรือ M005 รหัสนี้ใช้เชื่อมระหว่างนิยาม KPI กับค่าที่คำนวณ และใช้เป็นตัวกรองใน Metabase

7.1 สร้าง dbt/seeds/metric_definition.csv

```
metric_key,metric_name,metric_label,description,formula,unit
M001,total_revenue,Total Revenue,"รายได้รวมจาก payment ที่ aggregate ต่อ rental แล้ว",SUM(revenue_amount),currency
M002,rental_count,Rental Count,"จำนวนการเช่าภาพยนตร์",SUM(rental_count),count
M003,active_customer_count,Active Customer Count,"จำนวนลูกค้าที่มี rental แบบไม่ซ้ำ",COUNT_DISTINCT(customer_id),count
M004,average_revenue_per_rental,Average Revenue per Rental,"รายได้เฉลี่ยต่อการเช่าหนึ่งครั้ง",M001/M002,currency_per_rental
M005,late_return_rate,Late Return Rate,"สัดส่วนรายการคืนล่าช้าต่อรายการที่คืนแล้ว",late_rental_count/returned_rental_count,ratio
```

การบันทึกไฟล์ CSV: ให้บันทึกเป็น UTF-8 และอย่าเพิ่มช่องว่างก่อนหรือหลัง metric_key เพราะ M001 กับ M001[space] จะถือเป็นคนละค่า

7.2 โหลด Seed เข้า PostgreSQL

```
dbt seed --full-refresh
```

คำอธิบาย:

- dbt seed อ่านไฟล์ CSV และสร้างเป็นตารางในฐานข้อมูล
- --full-refresh สร้างตาราง Seed ใหม่ทั้งหมด ทำให้การแก้ไขนิยามใน CSV ถูกอัปเดตแน่นอน
- ผลลัพธ์จะเป็นตาราง dbt_metadata.metric_definition
- การเก็บนิยาม KPI ในไฟล์ CSV ทำให้สามารถ version control และตรวจสอบการเปลี่ยนแปลงได้

ส่วนที่ 8: สร้าง Metric Model ระดับบริษัท

8.1 สร้าง dbt/models/marts/metric_company_monthly.sql

```

with monthly_base as (
  select
    rental_month as metric_month,
    sum(revenue_amount) as total_revenue,
    sum(rental_count) as rental_count,
    count(distinct customer_id) as active_customer_count,
    sum(late_rental_count) as late_rental_count,
    sum(returned_rental_count) as returned_rental_count
  from {{ ref('fct_rental_activity') }}
  group by rental_month
),

metric_values as (
  select
    metric_month,
    'M001'::varchar as metric_key,
    total_revenue::numeric(18, 4) as metric_value
  from monthly_base

  union all

  select
    metric_month,
    'M002',
    rental_count::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
    'M003',
    active_customer_count::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
    'M004',
    (
      total_revenue
      / nullif(rental_count, 0)
    )::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
    'M005',
    (
      late_rental_count::numeric
      / nullif(returned_rental_count, 0)
    )::numeric(18, 4)
  from monthly_base
)

select
  v.metric_month,
  v.metric_key,
  d.metric_name,
  d.metric_label,
  d.description,
  d.formula,
  d.unit,
  v.metric_value
from metric_values v
join {{ ref('metric_definition') }} d

```

```
on v.metric_key = d.metric_key
```

คำอธิบาย:

- **monthly_base** คำนวณตัวตั้งและตัวหารจาก Fact ที่ Grain ระดับ rental ก่อน
- **UNION ALL** เปลี่ยนข้อมูลจากแบบหลายคอลัมน์เป็นแบบ Long Format โดยให้แต่ละ KPI อยู่คนละแถวและระบุด้วย metric_key
- **M004** คำนวณจาก total_revenue หาร rental_count ไม่ใช่ AVG ของค่าเฉลี่ยรายสาขา
- **M005** ใช้ NULLIF ป้องกันการหารด้วยศูนย์ในเดือนที่ยังไม่มีรายการคืน
- Join กับ metric_definition ทำให้ผลลัพธ์มีชื่อ คำอธิบาย สูตร และหน่วยวัดสำหรับ Metabase

ส่วนที่ 9: สร้าง Metric Model ระดับสาขา

9.1 สร้าง dbt/models/marts/metric_store_monthly.sql

```
with monthly_base as (
  select
    rental_month as metric_month,
    store_id,
    sum(revenue_amount) as total_revenue,
    sum(rental_count) as rental_count,
    count(distinct customer_id) as active_customer_count,
    sum(late_rental_count) as late_rental_count,
    sum(returned_rental_count) as returned_rental_count
  from {{ ref('fct_rental_activity') }}
  group by rental_month, store_id
),

metric_values as (
  select
    metric_month,
    store_id,
    'M001'::varchar as metric_key,
    total_revenue::numeric(18, 4) as metric_value
  from monthly_base

  union all

  select
    metric_month,
    store_id,
    'M002',
    rental_count::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
    store_id,
    'M003',
    active_customer_count::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
    store_id,
    'M004',
    (
      total_revenue
      / nullif(rental_count, 0)
    )::numeric(18, 4)
  from monthly_base

  union all

  select
    metric_month,
```

```

        store_id,
        'M005',
        (
            late_rental_count::numeric
            / nullif(returned_rental_count, 0)
        )::numeric(18, 4)
    from monthly_base
)

select
    v.metric_month,
    v.store_id,
    v.metric_key,
    d.metric_name,
    d.metric_label,
    d.description,
    d.formula,
    d.unit,
    v.metric_value
from metric_values v
join {{ ref('metric_definition') }} d
on v.metric_key = d.metric_key

```

Model นี้ใช้สูตรเดียวกับระดับบริษัท แต่เพิ่ม store_id ใน Grain จึงใช้สำหรับเปรียบเทียบ KPI ระหว่างสาขา

ส่วนที่ 10: กำหนด Documentation และ Data Tests

10.1 สร้าง dbt/models/schema.yml

```

version: 2

models:
  - name: fct_rental_activity
    description: >
      Fact model ที่มีหนึ่งแถวต่อหนึ่ง rental_id
    columns:
      - name: rental_id
        tests:
          - not_null
          - unique
      - name: rental_month
        tests:
          - not_null
      - name: store_id
        tests:
          - not_null
      - name: revenue_amount
        tests:
          - not_null
      - name: rental_count
        tests:
          - not_null

  - name: metric_company_monthly
    description: >
      KPI รายเดือนระดับบริษัทในรูป Long Format
    columns:
      - name: metric_month
        tests:
          - not_null
      - name: metric_key
        tests:
          - not_null
          - relationships:
              to: ref('metric_definition')
              field: metric_key
      - name: metric_value
        tests:
          - not_null:
              config:
                where: "metric_key NOT IN ('M004', 'M005')"

  - name: metric_store_monthly

```

```

description: >
  KPI รายเดือนแยกตามสาขาในรูปแบบ Long Format
columns:
  - name: metric_month
    tests:
      - not_null
  - name: store_id
    tests:
      - not_null
  - name: metric_key
    tests:
      - not_null
      - relationships:
          to: ref('metric_definition')
          field: metric_key
  - name: metric_value
    tests:
      - not_null:
          config:
            where: "metric_key NOT IN ('M004', 'M005')"

seeds:
  - name: metric_definition
    description: >
      Catalog กลางสำหรับรหัสและนิยาม KPI
    columns:
      - name: metric_key
        tests:
          - not_null
          - unique
      - name: metric_name
        tests:
          - not_null

```

Generic Tests ที่ใช้:

- `not_null` ตรวจสอบว่าคอลัมน์สำคัญไม่มีค่า NULL
- `unique` ตรวจสอบว่า `rental_id` และ `metric_key` ใน Catalog ไม่ซ้ำ
- `relationships` ตรวจสอบว่า `metric_key` ใน Metric Model มีอยู่จริงใน `metric_definition`
- `tests` เหล่านี้ไม่ต้องติดตั้ง package เพิ่ม เพราะเป็นความสามารถพื้นฐานของ dbt

10.2 ทดสอบ Grain ของ `metric_company_monthly`

สร้าง `dbt/tests/assert_metric_company_grain.sql`

```

select
  metric_month,
  metric_key,
  count(*) as row_count
from {{ ref('metric_company_monthly') }}
group by metric_month, metric_key
having count(*) > 1

```

dbt Singular Test จะถือว่าผ่านเมื่อ Query ไม่คืนแถว หากคืนผลลัพธ์แสดงว่ามีมากกว่าหนึ่งแถวต่อเดือนและ `metric_key` ซึ่งผิด Grain

10.3 ทดสอบ Grain ของ `metric_store_monthly`

สร้าง `dbt/tests/assert_metric_store_grain.sql`

```

select
  metric_month,
  store_id,
  metric_key,
  count(*) as row_count
from {{ ref('metric_store_monthly') }}
group by metric_month, store_id, metric_key
having count(*) > 1

```

10.4 ทดสอบช่วงค่าของ Late Return Rate

สร้าง `dbt/tests/assert_late_return_rate_range.sql`

```
select
  'company'::varchar as source_model,
  metric_month,
  null::integer as store_id,
  metric_value
from {{ ref('metric_company_monthly') }}
where metric_key = 'M005'
   and (metric_value < 0 or metric_value > 1)

union all

select
  'store'::varchar as source_model,
  metric_month,
  store_id,
  metric_value
from {{ ref('metric_store_monthly') }}
where metric_key = 'M005'
   and (metric_value < 0 or metric_value > 1)
```

Late Return Rate ต้องอยู่ระหว่าง 0 และ 1 หาก Query ค้นแถวอาจเกิดจากสูตร ตัวหาร หรือ Grain ไม่ถูกต้อง

ส่วนที่ 11: Build และตรวจสอบผลลัพธ์

11.1 สร้างทุก Model และรัน Test

```
dbt build
```

`dbt build` จะดำเนินงานตาม dependency โดยอัตโนมัติ ได้แก่ โหลด Seed สร้าง Staging สร้าง Intermediate สร้าง Fact สร้าง Metric Models และรัน Tests หากขั้นตอนใดไม่ผ่าน `dbt` จะหยุดหรือข้ามงานปลายทางที่ขึ้นต่อกัน

11.2 ตรวจสอบ Metric Catalog

```
SELECT *
FROM dbt_metadata.metric_definition
ORDER BY metric_key;
```

11.4 ตรวจสอบ KPI ระดับบริษัท

```
SELECT
  metric_month,
  metric_key,
  metric_label,
  metric_value,
  unit
FROM dbt_metrics.metric_company_monthly
ORDER BY metric_month, metric_key
LIMIT 20;
```

11.5 ตรวจสอบ KPI ระดับสาขา

```
SELECT
  metric_month,
  store_id,
  metric_key,
  metric_label,
  metric_value
FROM dbt_metrics.metric_store_monthly
ORDER BY metric_month, store_id, metric_key
LIMIT 30;
```

จุดตรวจสอบ: ในหนึ่งเดือน metric_company_monthly ควรมีไม่เกิน 5 แถว คือ M001-M005 และ metric_store_monthly ควรมีไม่เกิน 5 แถวต่อหนึ่งสาขา หากมากกว่านี้ให้ตรวจ GROUP BY และ JOIN

ส่วนที่ 12: นำ Metric Key ไปใช้ใน Metabase

12.1 เปิด Metabase และเชื่อมต่อฐานข้อมูล

เปิดเว็บเบราว์เซอร์ที่ <http://localhost:23000>

หากยังไม่ได้เชื่อมต่อ dvdrental ให้เพิ่ม PostgreSQL database ด้วยค่าต่อไปนี้

รายการ	ค่า
Database type	PostgreSQL
Display name	dvdrental
Host	postgres
Port	5432
Database name	dvdrental
Username	dw_user
Password	dw_pass

กรณีเชื่อมต่อจากโปรแกรมบนเครื่องโดยตรง: pgAdmin ที่เปิดจาก browser ผ่าน container ใช้ Host postgres ได้เมื่อเชื่อมผ่าน Docker network แต่เครื่องมือ PostgreSQL ที่ติดตั้งบนเครื่อง host ต้องใช้ localhost และ port 25432 ตาม docker-compose.yaml

12.2 Sync Schema

1. ไปที่ Admin settings → Databases → dvdrental
2. กด Sync database schema now
3. ตรวจสอบว่า Metabase มองเห็น schema dbt_metrics และ dbt_metadata
4. หากยังไม่พบ ให้รอการ Sync สักครู่แล้ว Refresh หน้า Browser

12.3 สร้าง Question: KPI Trend by Metric Key

1. เลือก New → Question → dvdrental → dbt_metrics → metric_company_monthly
2. เพิ่ม Filter ที่ metric_key และเลือก M001
3. เลือก Visualization เป็น Line Chart
4. กำหนด X-axis = metric_month และ Y-axis = metric_value
5. บันทึกชื่อ KPI Trend by Metric Key

Question นี้ใช้ metric_key เป็นตัวเลือกสูตร KPI โดย Metabase ไม่ต้องเขียน SUM, COUNT DISTINCT หรือสูตรหารใหม่ เพราะ dbt คำนวณไว้แล้ว

12.4 สร้าง Question: KPI by Store

1. เลือกตาราง dbt_metrics.metric_store_monthly
2. Filter metric_key = M001
3. เลือก Visualization เป็น Bar Chart
4. กำหนด X-axis = store_id และ Y-axis = metric_value
5. บันทึกชื่อ KPI by Store

12.5 สร้าง Question: Late Return Rate by Store

1. Duplicate Question KPI by Store
2. ตั้งชื่อ Late Return Rate by Store
3. เปลี่ยน Filter metric_key เป็น M005
4. ใน Formatting ตั้งค่า metric_value เป็น Percent

12.6 สร้าง Question: Metric Catalog

1. เลือกตาราง dbt_metadata.metric_definition
2. แสดงคอลัมน์ metric_key, metric_label, description, formula และ unit
3. เลือก Visualization เป็น Table
4. บันทึกชื่อ Metric Catalog

12.7 สร้าง Dashboard

1. สร้าง Dashboard ชื่อ DVD Rental KPI Dashboard - [รหัสนักศึกษา]
2. เพิ่ม Question ทั้ง 4 รายการลง Dashboard
3. เพิ่ม Dashboard Filter แบบ Dropdown ชื่อ Metric Key และเชื่อมกับ metric_key ของ KPI Trend และ KPI by Store
4. เพิ่ม Date Filter เชื่อมกับ metric_month
5. จัดวางกราฟให้ชื่อ KPI, หน่วย และช่วงเวลามองเห็นชัดเจน

ข้อควรระวัง: Metric Key แต่ละตัวมีหน่วยต่างกัน เช่น currency, count และ ratio การเปลี่ยน metric_key ในกราฟเดียวกันอาจทำให้รูปแบบแกน Y ไม่เปลี่ยนอัตโนมัติ จึงควรแสดง unit ในตารางหรือสร้างกราฟเฉพาะสำหรับ KPI ที่ต้องใช้รูปแบบเปอร์เซ็นต์

ส่วนที่ 13: คำถามวิเคราะห์

1. เพราะเหตุใดจึงต้อง Aggregate payment ให้เหลือหนึ่งแถวต่อ rental_id ก่อน Join กับ rental?
2. เพราะเหตุใด Active Customer Count รายปีจึงไม่ควรคำนวณด้วยการบวก Active Customer Count ของแต่ละเดือน?
3. ถ้าเปลี่ยนนิยาม Late Return เป็น “คืนเกินกำหนดอย่างน้อย 2 วัน” ต้องแก้ไขไฟล์ใด และต้องรันคำสั่งใดอีกครั้ง?
4. หากให้ผู้เขียนสูตร M004 และ M005 เองทุก Dashboard จะเกิดความเสี่ยงอย่างไร?

สิ่งที่ต้องส่ง

รายการ	รูปแบบ
Screenshot Metabase Dashboard	PNG / JPG
คำตอบคำถามวิเคราะห์ 4 ข้อ	พิมพ์ใน Google Form

สรุปแนวคิดของ Lab

ลำดับการทำงานของ Lab นี้คือ Requirement → Business Process → KPI → Grain → Fact Model → Metric Key Catalog → Metric Models → dbt Tests → Metabase Dashboard

dbt ทำหน้าที่เป็นจุดกลางของ Business Logic ส่วน Metabase ทำหน้าที่นำเสนอข้อมูล นักศึกษาจึงไม่ควรสร้างสูตร KPI ซ้ำใน Metabase เพราะการแก้สูตรใน dbt เพียงจุดเดียวจะทำให้ทุก Dashboard ที่ใช้ View เดียวกันได้รับนิยามใหม่อย่างสอดคล้องกัน